
Paradigmas de Programación

EIF400-II-2026- CLoria

Escuela de Informática – Universidad Nacional (UNA)

Proyecto Grupal: *Proyecto Cyphail Sprint P1*

Versión 1 Setiembre-2026 Preliminar

I. Contexto

Este `SPEC` es en el marco del proyecto `Cyphail` del curso `EIF400-II-2026-CLoria`. Se proponen los nuevos features a incorporar al proyecto en el sprint denominado `P1`. Se entiende en un contexto ágil y dinámico que puede requerir ajustes.

Son **expectativas obligatorias**:

- Se asume que el previo Sprint `P1.1` ya ha alcanzado, es flexible y permite iterarse para conseguir lo acá planteado. Caso contrario se debe adaptar según corresponda.
- Se trabaja con el mismo modelo y lenguaje (Java) de proyecto, actualizando todo lo que corresponda (readme, tests, documentación, build, deploy a github y defensa).
- Todo lo que funcionaba antes debe seguir funcionando (no hay regresiones).
- Al revisar el historial de commits se aprecia trabajo más allá de *tres días antes de la entrega* y participación de todo el equipo (push temprano y continuo).

Fechas y modo de entrega:

Se definirán oportunamente

II. Nuevos Features

1. Hay Parsing: Se parsean estatutos (queries) simples del tipo `MATCH-WHERE-RETURN` y se genera el `AST` como en el siguiente ejemplo

```
MATCH (m:Movie)
WHERE m.year > 1990
RETURN m.title AS title,
       m.year AS year
```

2. Habría un nuevo comando del `repl` que permite (pretty) imprimir el `AST` en un estilo `JSON`

```

cyphail repl
>>>.tree
MATCH (m:Movie)
WHERE m.year > 1990
RETURN m.title AS title,
       m.year AS year

```

Con una posible salida comparable con la siguiente

```

Query{
  Match: {
    Patterns: [
      PatternNode:{
        var: m
        labels: [ Movie ]
        properties: []
      }
    ]
  }
  Where: {
    Expr: (> (. m year) 1990)
  }
  Updates:[]
  Return:{
    Projection:{
      Items:[
        {as (. m title) title)}
        {as (. m year) year)}
      ]
      Modifiers:[]
    }
  }
}

```

Como se observa, hay elementos de los queries que no se requieren aún como properties ni relaciones de grafos. Las expresiones se muestran en estilo pre-order.

Modelo de `AST` y validación básica

- Este feature requiere crear un modelo de `AST` que el parser construye. Una visita al `AST` particular es la que permite la salida en el `repl` asociada con `.tree`.

- Si al generar la salida se detectan variables en expresiones o proyecciones no definidas en los patrones se dan mensajes (en inglés) reportando tal problema. Note en ejemplo, que `title` en `m.title`, por ejemplo, **no sería una variable no definida**, pues es un property de la variable `m` que sí está definida.

II. Paradigma

1. Parser de combinadores propio siguiendo modelo funcional estudiado en clase migrado a `Java`
2. En la parte del `repl` pueden usar programación imperativa en temas que son naturalmente imperativos.

III Engine

No existe aún del todo.

IV Fake Broker (No alambramiento)

- Todo el fake engine y broker que en `P1.1` era alambrado en el código, sigue siendo fake, pero ahora se lee de disco los datos de disco en formato `JSON`. Tengan una carpeta `data` en el proyecto donde se puedan ver los archivos que sirven para este propósito. Una implicación obvia es lectura en disco.
- Durante la defensa se podrían cambiar los datos fake sin tener que recompilar la aplicación y ver los cambios reflejados en el `repl`. Debe haber un `JSON` en disco con la fake data asociada a cada query de testing que el parser parsea.
- La implicación es incorporar una librería de manejo de `JSON` en el proyecto (por ejemplo, `GSON`).
- Este cambio debe permitir la conexión final en Sprint `P2` de `JSON` que vendrían de `PROLOG`.